

ICT1011

Computer Organisation and Architecture

Chapter 5

Executable Assembly Instructions

Executable Assembly Instructions

1. MSP430 Instruction Set and Emulated Instructions

2. Executable Instructions

- Data Movement
- Arithmetic
- Logical and Bit Manipulation
- Rotation
- Program Control

Introduction to Assembly Language Programming

1. MSP430 Instruction Set and Emulated Instructions

2. Executable Instructions

- Data Movement
- Arithmetic
- Logical and Bit Manipulation
- Rotation
- Program Control



Instructions and Emulated Instructions

- The MSP430 has 27 Core Instructions
 - Opcodes of these instructions can be converted to machine code
- In addition, the MSP430 supports another 24 Emulated Instructions
 - These opcodes are emulated by one of the Core Instruction
 - Emulated Instruction does not add any new functionality, but makes coding easier.

Mnemonic	Operation	Emulation	Description
Arithmetic Instructions			
DEC(.B or .W) dst	dst-1→dst	SUB(.B or .W) #1,dst	Decrement destination
DECD(.B or .W) dst	dst-2→dst	SUB(.B or .W) #2,dst	Decrement destination twice
INC(.B or .W) dst	dst+1→dst	ADD(.B or .W) #1,dst	Increment destination
INCD(.B or .W) dst	dst+2→dst	ADD(.B or .W) #2,dst	Increment destination twice

Introduction to Assembly Language Programming

1. MSP430 Instruction Set and Emulated Instructions

2. Executable Instructions

- Data Movement
- Arithmetic
- Logical and Bit Manipulation
- Rotation
- Program Control



Data Movement Instructions

MOV

- The **mov** instruction provides data movement between a source and a destination.
- The mnemonic is given by **mov.x src, dst**.
- The suffix **.x** can be byte (**.b**) or word (**.w**).
- Source operand is not modified by the execution.
- Status Bits are not affected:

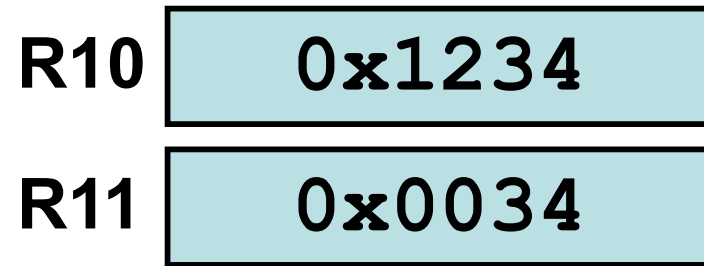
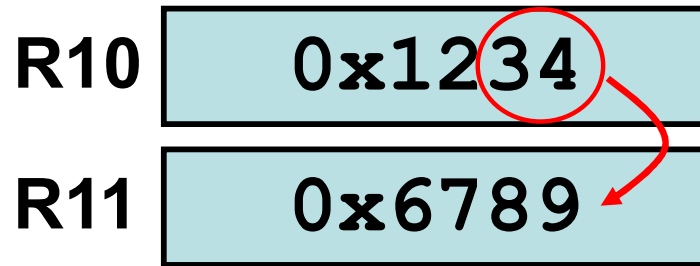
			Status Bits			
			V	N	Z	C
MOV(.B)	src, dst	src → dst	-	-	-	-

This information can be obtained from the **Instruction Set Summary** or the **Instruction Set Description** documents

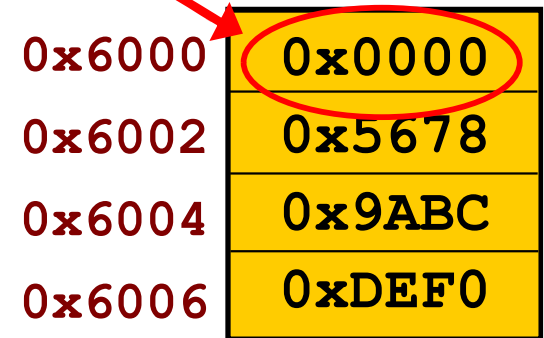
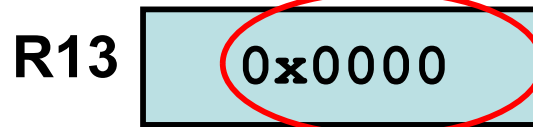
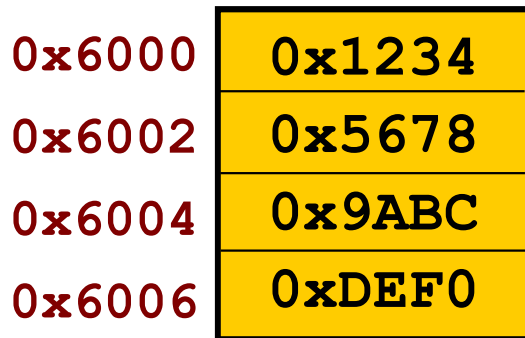
Data Movement Instructions

MOV - Example

Example: **mov.b R10, R11**



Example: **mov.w R13, &0x6000**



Introduction to Assembly Language Programming

1. MSP430 Instruction Set and Emulated Instructions

2. Executable Instructions

- Data Movement
- Arithmetic
- Logical and Bit Manipulation
- Rotation
- Program Control



Arithmetic Instructions

SXT

SXT sign-extends the sign bit of the lower-order byte in a register to the remaining higher-order bits (copies bit 7 into bits 8-15).

The mnemonic is given by **sxt reg**

Status Bits	N: Set if result is negative, reset if positive
	Z: Set if result is zero, reset otherwise
	C: Set if result is not zero, reset otherwise (.NOT. Zero)
	V: Reset

e.g. **sxt R10** ; copies bit 7 in R10 into upper bits 8 to 15 of R10

R10 **0x1284**

N=0, Z=0, V=1, C=0

Before execution

R10 **0xFF84**

N=1, Z=0, V=0, C=1

After execution

Arithmetic Instructions

ADD

- Adds source operand to the destination operand.
- The mnemonic is given by **add.x src, dst**.
- The suffix **.x** can be a byte (**.b**) or word (**.w**)
- Result is stored in the destination.

Status Bits	N: Set if result is negative, reset if positive
	Z: Set if result is zero, reset otherwise
	C: Set if there is a carry from the result, cleared if not.
	V: Set if an arithmetic overflow occurs, otherwise reset

- e.g. **add.b R10, R11** ; add byte contents in R10 and R11
; MSB of R11 is cleared to 0x00
- e.g. **add.w #2, R13** ; add 2 to the contents of R13

Program Statement - Addition

- Write an assembly language program to :
- Add the contents of two 16-bit variables stored in consecutive memory locations.
- Store the result in the third memory location immediately following the two values.
- Getting started:
 - Determine the data and code areas in memory, e.g. **.data** section for the variable, **.text** section for the code
 - Determine the label names to give to the two numbers and result, e.g. **N1** and **N2** for numbers and **A1** for answer.

Program Solution - Addition

A simple program to illustrate the addition of two memory variables and the storing of its result to another memory variable.

```

        .data
N1      .space    2           ;allocate 1 word for variable N1
N2      .space    2           ;allocate 1 word for variable N2
A1      .space    2           ;allocate 1 word for variable A1

        .text
mov.w   &N1, R10              ;move word at N1 to register R10
mov.w   &N2, R11              ;move word at N2 to register R11
add.w   R11, R10              ;add R10 with R11
mov.w   R10, &A1              ;save word result in R10 to variable A1

        .end

```

Can you find a quicker way to do this???

Arithmetic Instructions

ADDC

- Adds source operand and carry flag (**C**) to the content destination operand.
- The mnemonic is given by **addc.x src,dst**.
- The suffix **.x** can be a byte (**.b**) or word (**.w**)
- Result is stored in the destination.

Status Bits	N: Set if result is negative, reset if positive
	Z: Set if result is zero, reset otherwise
	C: Set if there is a carry from the MSB of the result, reset if not
	V: Set if an arithmetic overflow occurs, otherwise reset

● e.g. **addc.w R10,R11** ; R11 = R10 + R11 + C

● e.g. **addc.w #2,R13** ; R13 = R13 + 2 + C

Arithmetic Instructions

Use of ADDC

- ADDC is often used to support higher-order addition
- For example, write an assembly program to help you add two 32-bit numbers;

H'77778888 + H'55556666

and store the result in two registers, R11 storing bits 0-15 of the results and R12 storing bits 16-31 of the results?

```
mov.w    #0x7777, R12 ;store the higher 16-bits for 1st num
mov.w    #0x8888, R11 ;store the lower 16-bits for 1st num
add.w    #0x6666, R11 ;adds the lower 16-bits
addc.w   #0x5555, R12 ;add the upper 16-bits as well as the
                        C-flag from the previous operation
```

Arithmetic Instructions

DADD

- The source operand and the destination operand are treated as four **binary coded decimals (BCD)** with positive signs.
- The source operand and the carry C are added decimally to the destination operand.
- The mnemonic is given by **dadd.x src, dst.**
- The suffix **.x** can be a byte (**.b**) or word (**.w**)
- Result is stored in the destination.

Status Bits	N: Set if the MSB is 1, reset otherwise
	Z: Set if result is zero, reset otherwise
	C: Set if the result is greater than 9999. Set if the result is greater than 99.
	V: Undefined

ADD vs DADD (An Example)

Example: **add.w** R10, R11

R10	0x1234
R11	0x6789

Before execution

R10	0x1234
R11	0x79BD

After execution

Example: **dadd.w** R10, R11

R10	0x1234
R11	0x6789

Before execution

R10	0x1234
R11	0x8023

After execution

DADD.W treats the contents as decimal numbers.
ADD.W treats the contents as binary numbers.

Arithmetic Instructions

SUB

The source operand is subtracted from the destination operand.

This is made by adding the 2's complement of the source operand to the destination operand. The status bits are set/cleared based on this addition operation

$$\text{dst} + (-(\text{src})) \rightarrow \text{dst}$$

Status Bits	N: Set if result is negative, reset if positive
	Z: Set if result is zero, reset otherwise
	C: Set if there is a carry from the result, cleared if not.
	V: Set if an arithmetic overflow occurs, otherwise reset

The mnemonic is given by **sub.x src, dst**.

The suffix **.x** can be a byte (**.b**) or word (**.w**)

e.g. **sub.b R10, R11** ; subtract byte contents in R10 and R11

; MSB of R11 will be cleared to 0x00

SUBC

- Subtracts source operand and carry flag (**C**) from content of the destination operand.
- Similar to **addc**, **subc** is used to support subtraction of numbers that are >16-bits.
- CCR flags are affected as in the **sub.b** instruction
- e.g. **subc.b R10,R11** ; R11 = R11 - R10 - C
- e.g. **subc #5,R12** ; R12 = R12 - 5 - C

Use of SUBC

- Like ADDC, SUBC is often used to support higher-order addition

- For example, write an assembly program to help you subtract two 32-bit numbers;

H'77778888 - H'55556666

and store the result in two registers, R11 storing bits 0-15 of the results and R12 storing bits 16-31 of the results?

```

mov.w      #0x7777, R12 ;store the higher 16-bits for 1st num
mov.w      #0x8888, R11 ;store the lower 16-bits for 1st num
sub.w      #0x6666, R11 ;subtract the lower 16-bits
subc.w     #0x5555, R12 ;subtract the upper 16-bits as well as
                        the C-flag from the previous operation
    
```

Emulated Arithmetic Instructions

7 additional arithmetic instructions can be emulated using the core arithmetic instructions.

Mnemonic	Operation	Emulation	Description
Arithmetic Instructions			
ADC(.B or .W) dst	dst+C→dst	ADDC(.B or .W) #0,dst	Add carry to destination
DADC(.B or .W) dst	dst+C→dst (decimally)	DADD(.B or .W) #0,dst	Decimal add carry to destination
DEC(.B or .W) dst	dst-1→dst	SUB(.B or .W) #1,dst	Decrement destination
DECD(.B or .W) dst	dst-2→dst	SUB(.B or .W) #2,dst	Decrement destination twice
INC(.B or .W) dst	dst+1→dst	ADD(.B or .W) #1,dst	Increment destination
INCD(.B or .W) dst	dst+2→dst	ADD(.B or .W) #2,dst	Increment destination twice
SBC(.B or .W) dst	dst+0FFFFh+C→dst dst+0FFh→dst	SUBC(.B or .W) #0,dst	Subtract source and borrow /.NOT. carry from dest.

Introduction to Assembly Language Programming

1. MSP430 Instruction Set and Emulated Instructions

2. Executable Instructions

- Data Movement
- Arithmetic
- Logical and Bit Manipulation
- Rotation
- Program Control



Logical and Bit Manipulation

Logical Operations

Like most CPUs, logical operation between the source and destination operands can be computed and the result stored into the destination.

- 3 logical operations are supported by the MSP430 core instructions: **and**, **bis** (same as logical OR) and **xor**.
- The way that the status bits are affected varies with the logical operation used.
- The source operand can either be a register or an immediate value.

Logical and Bit Manipulation

and, bis and xor

- Logical instructions are used for the following purpose:
 - and** – **clear** specific bits in the destination register.
 - bis** – **set** specific bits in the destination register (it's a OR operation).
 - xor** – **complement** specific bits in the destination register.

AND truth table

A	B	Z = A . B
0	0	0
0	1	0
1	0	0
1	1	1

A logical 0 is used to clear

OR truth table

A	B	Z = A+B
0	0	0
0	1	1
1	0	1
1	1	1

A logical 1 is used to set

XOR truth table

A	B	Z = A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

A logical 1 is used to complement

Instruction examples – and, bis, xor

Bits 7 6 5 4 3 2 1 0
R10 B' 01010101

N=0, Z=0, V=1, C=0

Initial conditions of byte-sized register R10 and CCR flags

N: Set if MSB of result is set, reset if not set

Z: Set if result is zero, reset otherwise

C: Set if result is not zero, reset otherwise (= .NOT. Zero)

V: Reset

e.g. **and.b #11110000b, R10**

R10 B' 01010000

N=0, Z=0, V=0, C=1

Bit 0 to 3 cleared after execution

e.g. **bis.b #11110000b, R10**

Bis do not change the status bits

R10 B' 11110101

N=0, Z=0, V=1, C=0

Bit 4 to 7 set after execution

e.g. **xor.b #11110000b, R10**

N: Set if MSB of result is set, reset if not set

Z: Set if result is zero, reset otherwise

C: Set if result is not zero, reset otherwise (= .NOT. Zero)

V: Set if both operands are negative

R10 B' 10100101

N=1, Z=0, V=0, C=1

Bit 4 to 7 inverted after execution

Logical and Bit Manipulation

BIC

Other than the bit-set instruction **bis**, there is also a bit-clear instruction **bic**

The inverted source operand and the destination operand are logically AND'ed. The result is placed into the destination.

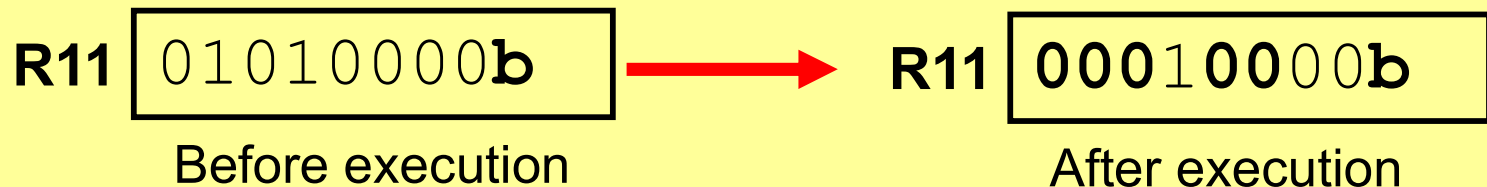
.NOT.src .AND. dst -> dst

The mnemonic is given by **bic.x src,dst**.

The suffix **.x** can be a byte (**.b**) or word (**.w**)

The status flags are no affected.

e.g. **bic.b #11101100b,R11**



Emulated Bit Manipulation Instructions

Other than the core instruction, the programmer can also modify certain bits (e.g. the status bits) using the emulated instructions.

Mnemonic	Operation	Emulation	Description
Data Instructions			
CLR(.B or .W) dst	0→dst	MOV(.B or .W) #0,dst	Clear destination
CLRC	0→C	BIC #1,SR	Clear carry flag
CLRN	0→N	BIC #4,SR	Clear negative flag
CLRZ	0→Z	BIC #2,SR	Clear zero flag
INV(.B or .W) dst	.NOT.dst→dst	XOR(.B or .W) #0(FF)FFh,dst	Invert bits in destination
SETC	1→C	BIS #1,SR	Set carry flag
SETN	1→N	BIS #4,SR	Set negative flag
SETZ	1→Z	BIS #2,SR	Set zero flag

Introduction to Assembly Language Programming

1. MSP430 Instruction Set and Emulated Instructions

2. Executable Instructions

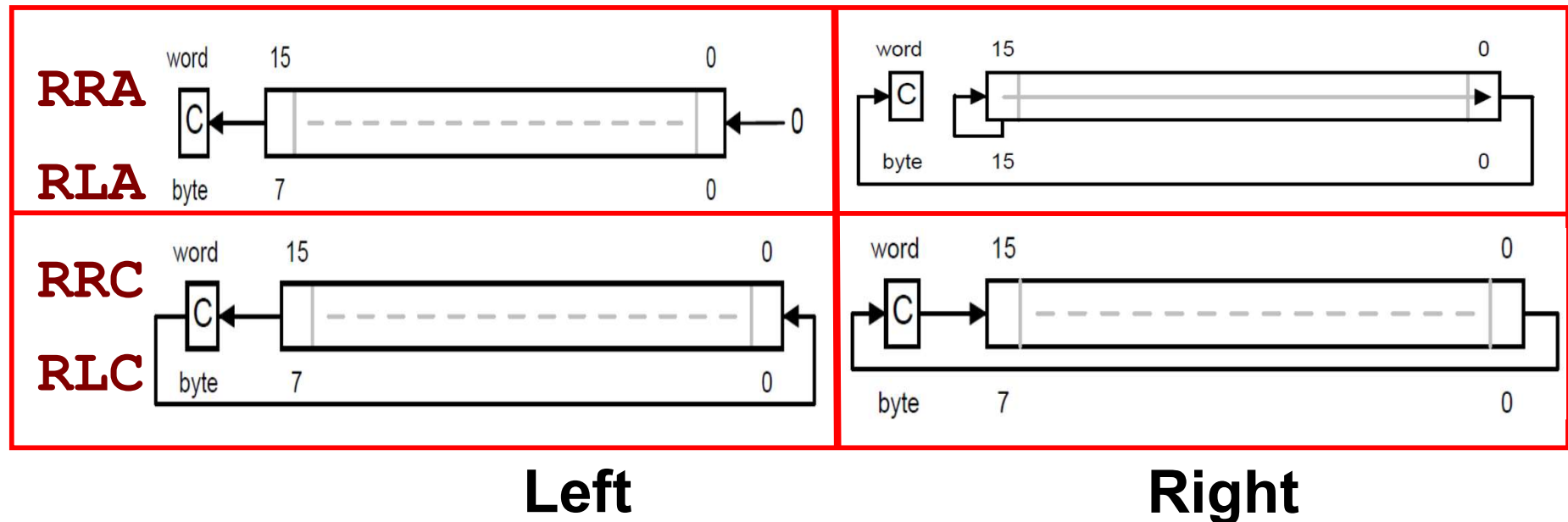
- Data Movement
- Arithmetic
- Logical and Bit Manipulation
- Rotation
- Program Control



Rotation Instructions

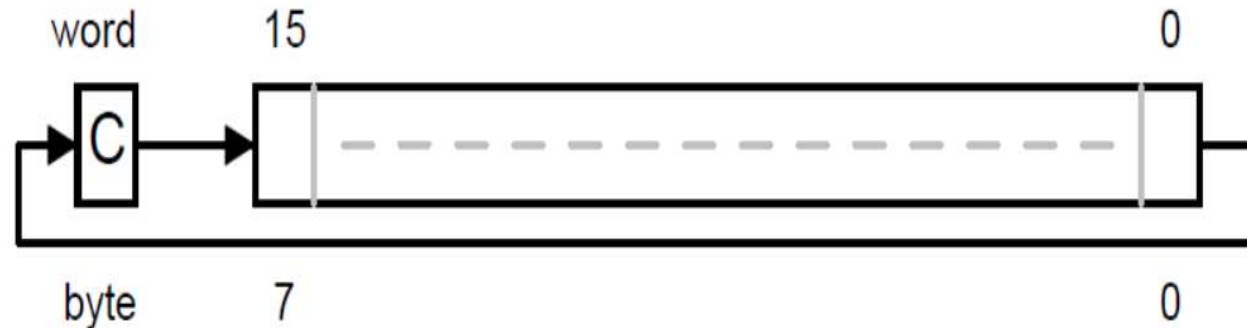
Summary of Rotation Instructions

- Rotation instructions are used to rotate bits within the destination operand
- 2 types of rotations: Arithmetic or “Through Carry”
- A visual summary of the operation for all the shift and rotate instructions:



Rotate Right through Carry RRC

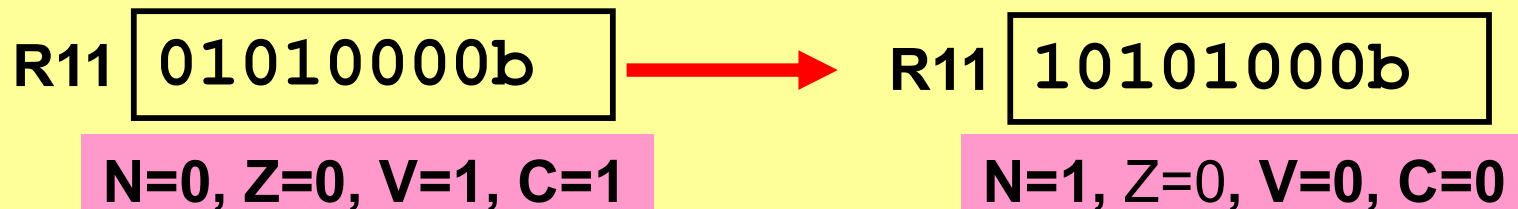
The destination operand is shifted right one position. The carry C is shifted into the MSB, the LSB is shifted into the carry C.



The mnemonic is given by **rrc.b dst** / **rrc.w dst**

N: Set if result is negative (MSB = 1), reset otherwise (MSB = 0)
Z: Set if result is zero, reset otherwise
C: Loaded from the LSB
V: Reset

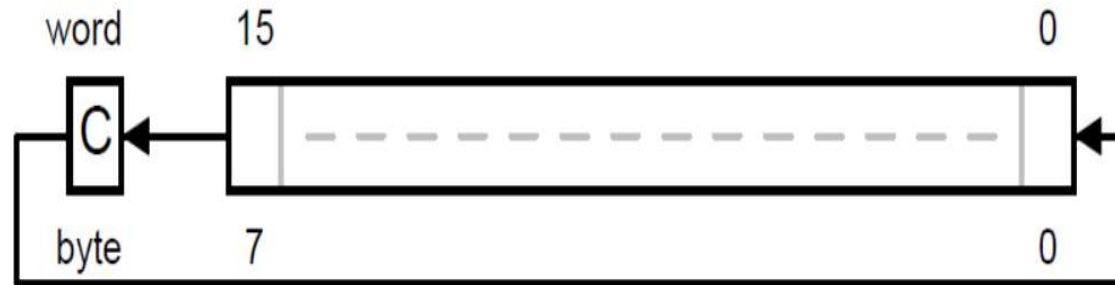
e.g. **rrc.b R11**



Rotate Left through Carry

RLC (emulated instruction using ADDC)

The destination operand is shifted left one position. The carry C is shifted into the LSB, the MSB is shifted into the carry C.



The mnemonic is given by **rlc.b dst** / **rlc.w dst**

Status Bits

Set when
change of
sign

N: Set if result is negative, reset if positive

Z: Set if result is zero, reset otherwise

C: Loaded from the MSB

V: Set if arithmetic overflow occurs otherwise reset

e.g. **rlc.b R11**

R11 **01010000b**

N=0, Z=0, V=0, C=1

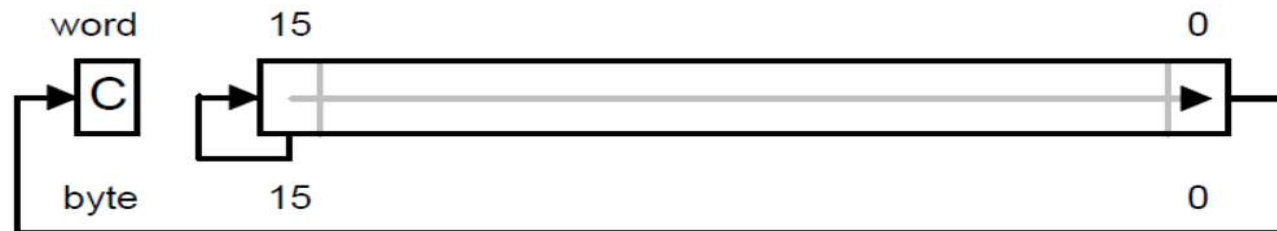
R11 **10100001b**

N=1, Z=0, V=1, C=0

Rotate Right Arithmetically

RRA

The destination operand is shifted right one position. The MSB is shifted into the MSB, the MSB is shifted into the MSB-1, the LSB+1 is shifted into the LSB.



The mnemonic is given by **rra.b dst** / **rra.w dst**

Status Bits	
N:	Set if result is negative, reset if positive
Z:	Set if result is zero, reset otherwise
C:	Loaded from the LSB
V:	Reset

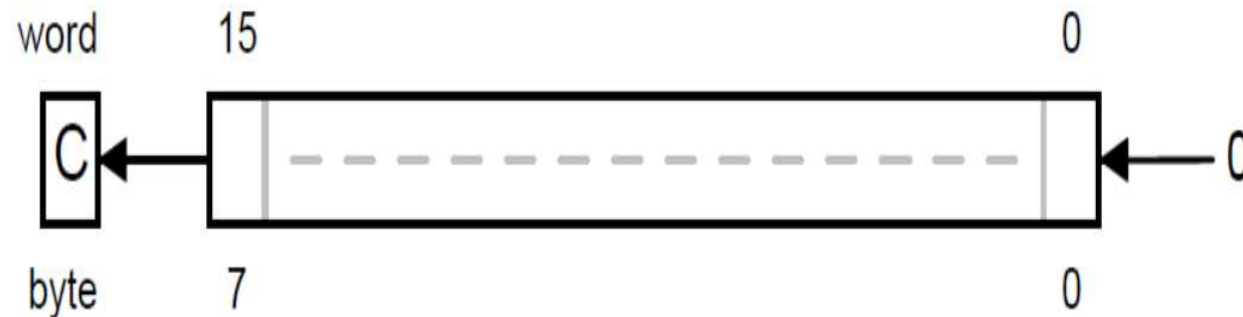
e.g. **rra.b R11**

R11	01010000b	→	R11	00101000b
	N=0, Z=0, V=1, C=1			N=0, Z=0, V=0, C=0

Rotate Left Arithmetically

RLA (emulated instruction using ADD)

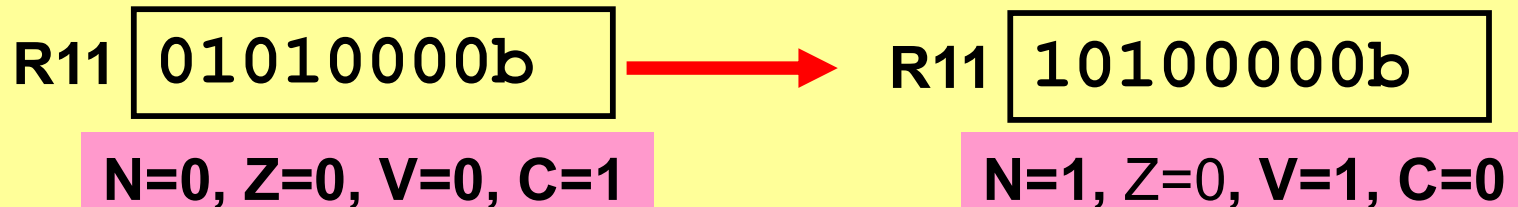
The destination operand is shifted left one position. The MSB is shifted into the carry C, the LSB is filled with 0.



The mnemonic is given by **rla.b dst** / **rla.w dst**

Status Bits	N: Set if result is negative, reset if positive
	Z: Set if result is zero, reset otherwise
	C: Loaded from the MSB
Set when change of sign →	V: Set if an arithmetic overflow occurs -

e.g. **rla.b R11**

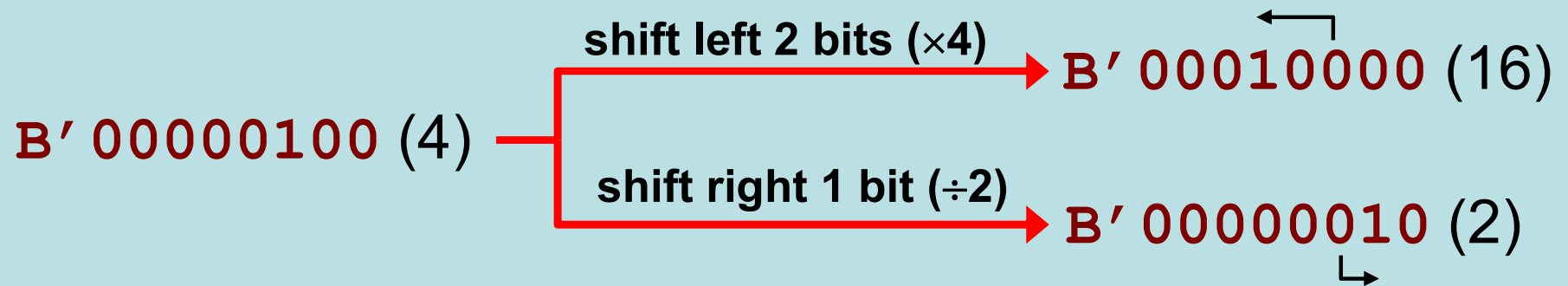


Rotation Instructions

Rotate to Multiply or Divide

- Arithmetic rotation performs multiply (shift left) or divide (shift right) by a factor of 2^N , where N is the no. of bits shifted.

e.g.

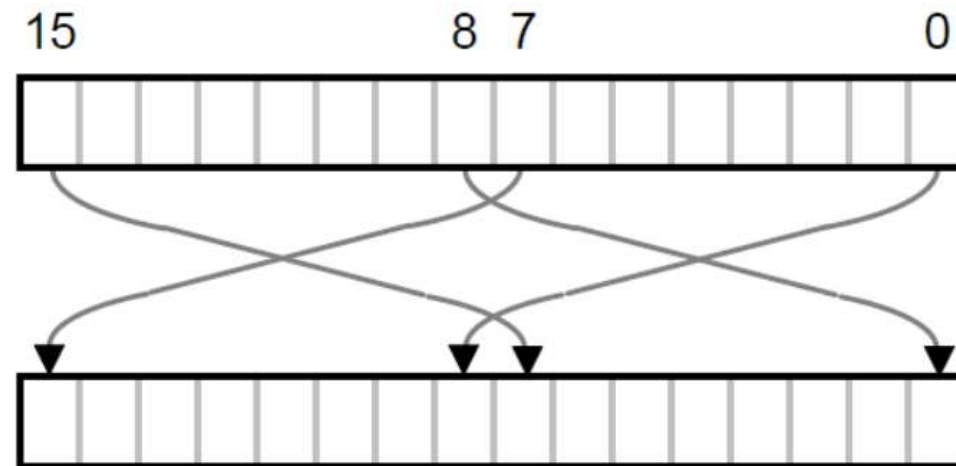


- RRA** → Supports **Division ($\div 2$)** by rotating bits one bit right
- RLA** → Supports **Multiplication ($\times 2$)** by rotating bits one bit left

Swap Bytes

SWPB

- The high and the low bytes of the destination operand are exchanged.
- The mnemonic is given by **swpb dst**
- Only supports word-sized operations, no status bits affected



e.g. **swpb R11**

R11 **0xAABB**



R11 **0xBBAA**

Introduction to Assembly Language Programming

1. MSP430 Instruction Set and Emulated Instructions

2. Executable Instructions

- Data Movement
- Arithmetic
- Logical and Bit Manipulation
- Rotation
- Program Control



Program Control Instructions

- These instructions facilitate the disruption of a program's normal sequential flow.
- The disruption of sequential flow is implemented using a jump or a conditional jump instruction.
- A jump can be obtained by modifying the contents of the Program Counter (**PC**).
- A jump can be executed based on a given condition (e.g. if result of previous execution is negative) and this is called a **conditional jump**.
- Conditional jump is useful for implementing:
 - conditional constructs (e.g. **if** or **if-else**)
 - loop constructs (e.g. **for** or **while** loops)

Program Control Instructions

Jump Instructions (recap)

- These 8 jump instructions are used to direct program flow to another part of the program.

Mnemonic	Description
Program flow control instructions	
JEQ/JZ label	Jump to label if zero flag is set
JNE/JNZ label	Jump to label if zero flag is reset
JC label	Jump to label if carry flag is set
JNC label	Jump to label if carry flag is reset
JN label	Jump to label if negative flag is set
JGE label	Jump to label if greater than or equal
JL label	Jump to label if less than
JMP label	Jump to label unconditionally

“label” is where the program will jump to if the conditions are met.

Program Control Instructions

Jump Instructions (recap)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code					Condition			10-bit, 2's complement PC offset							

- Jump instructions are used to direct program flow to another part of the program.
- The condition on which a jump occurs depends on the Condition field consisting of 3 bits:
 - 000: jump if not equal ($Z = 0$)
 - 001: jump if equal ($Z = 1$)
 - 010: jump if carry flag equal to zero ($C = 0$)
 - 011: jump if carry flag equal to one ($C = 1$)
 - 100: jump if negative ($N = 1$)
 - 101: jump if greater than or equal ($N = V$)
 - 110: jump if lower ($N \neq V$)
 - 111: unconditional jump

Program Control Instructions

Jump Instructions (recap)

- Jump instructions are executed based on the current PC and the status register
- Conditional jumps are controlled by the status bits
- Status bits are not changed by a jump instruction
- The jump off-set is represented by the 10-bit, 2's complement value:

$$PC_{new} = PC_{old} + 2 + PC_{offset} \times 2$$

PC_{old} is the address of the Jump instruction

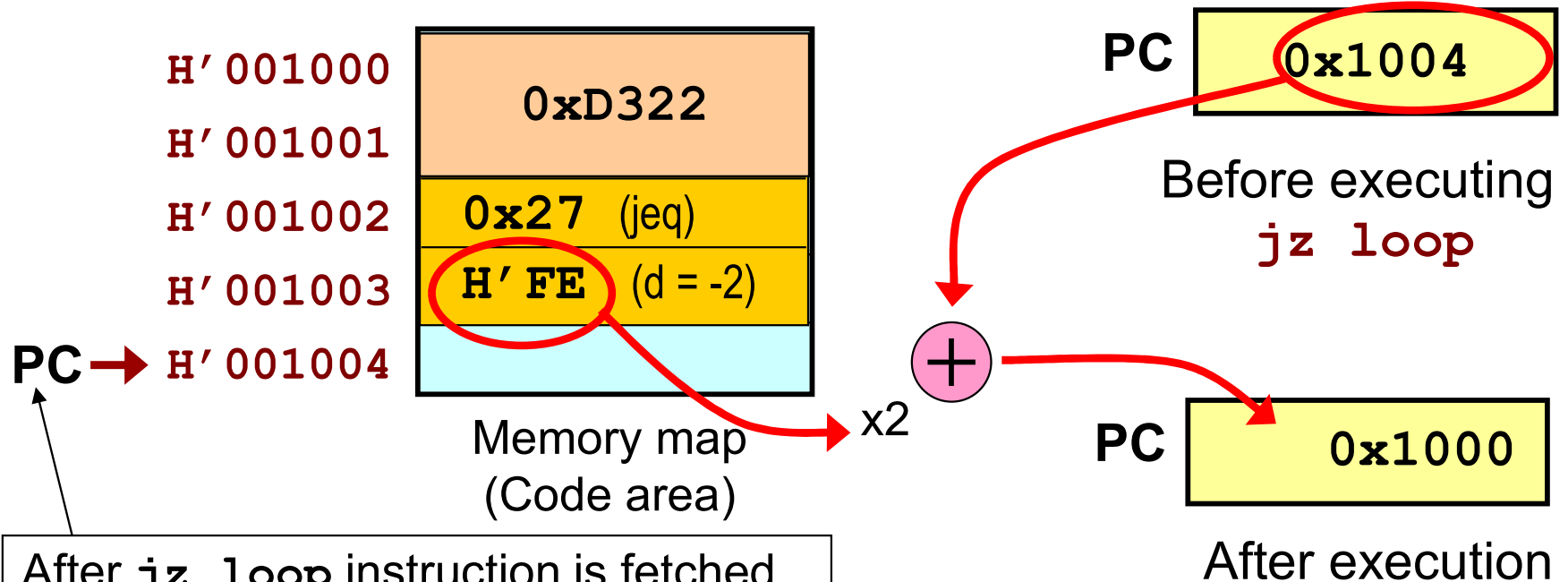
Program Control Instructions

Conditional jump example

E.g. `loop: SETZ`
`jz      loop`

Note: This is an example of a disastrous infinite loop

* jz and jeq are the same instruction



Implementing a simple count loop

- Consider the task of writing 0's to 10 bytes of memory starting at address 0x2400:

```

mov.w #10,R12           ;load 10 into counter register
mov.b #0,R10
loop mov.b R10,0x2400-1(R12)
      sub.w #1,R12       ;subtract 1 from counter register
      jne    loop        ;keep looping back 10 times
  
```

loop back
if result is
not zero

Continue sequential execution if result is zero
(i.e. exit count loop)

Implementation using register indirect with
pre-decrement and the **jne** conditional test

Program Control Instructions

Testing Unsigned and Signed Values

Appropriate conditional test must be selected based on the number representation used.

For testing **signed** values, use **JGE, JL, JN**

e.g.

```

cmp.w R12,R11
jge test ;jump to test if R12 is greater
:
test :
```

For testing **unsigned** values, use **JLO, JHS**

e.g.

```

cmp.w R2,R1
jlo test ;branch to R1>R2
:
R1>R2 :
```

Program Control Instructions

Bit Testing

- Bit testing can be used to control program flow;
- Hence, the CPU provides an instruction to test the state of individual or multiple bits;
- The operation performs a logical & (AND) operation between the source and destination operands;
- The result is ignored and none of the operands are changed.

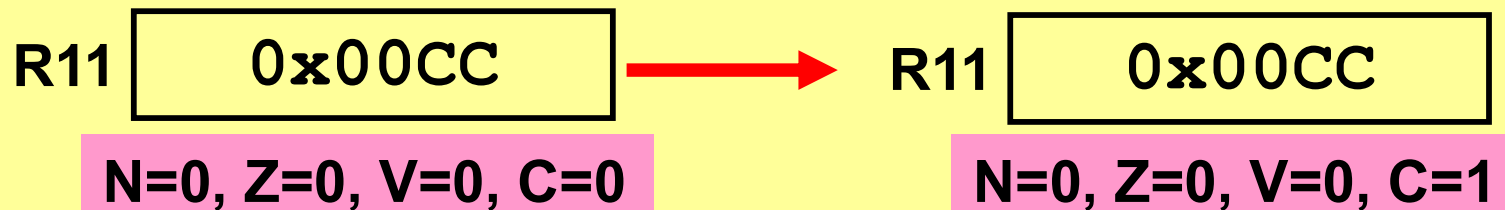
Program Control Instructions

BIT

- The source operand and the destination operand are logically AND'ed. The result affects only the Status Bits. ***The source and destination operands are not affected.***
- Usually used to check if certain bits of the destination are set/cleared before a conditional jump instruction.
- The mnemonic is given by **bit.x src,dst**

N: Set if MSB of result is set, reset if not set
Z: Set if result is zero, reset otherwise
C: Set if result is not zero, reset otherwise (.NOT. Zero)
V: Reset

• e.g. **bit.w #0x0080,R11**



Program Control Instructions

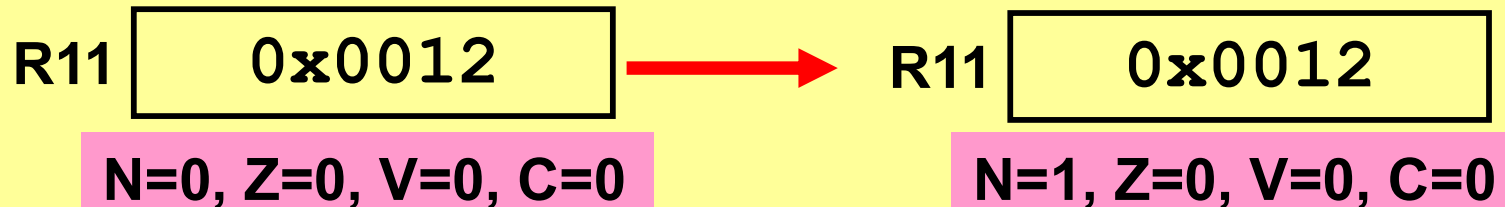
CMP

- The source operand is subtracted from the destination operand. **The two operands are not affected and, the result is not stored**; only the status bits are affected.
- Usually used to compare two values before a conditional jump instruction.
- The mnemonic is given by **cmp.x src, dst**

Status Bits	N: Set if result is negative, reset if positive (src >= dst)
	Z: Set if result is zero, reset otherwise (src = dst)
	C: Set if there is a carry from the MSB of the result, reset if not
	V: Set if an arithmetic overflow occurs, otherwise reset

$$0x0012 + (-0x0014) = 0x0012 + 0xFFEC = 0xFFFE$$

e.g. **cmp.w #0x0014, R11**



Program Control Instructions

TST (emulated instruction using cmp)

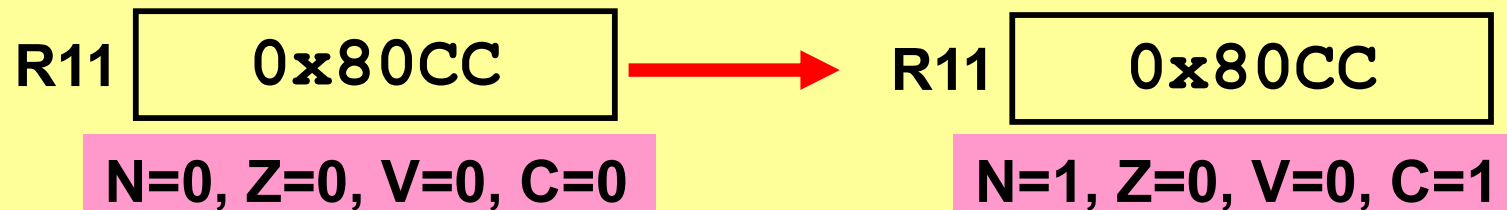
Mnemonic	Emulation	Description
TST(.B or .W) dst	CMP(.B or .W) #0,dst	Test destination

Used to test if a value is Neg or Zero. The destination operand is compared to zero. The status bits are set according to the result. **The destination is not affected.**

The mnemonic is given by **tst.x dst**

N: Set if destination is negative, reset if positive
Z: Set if destination contains zero, reset otherwise
C: Set
V: Reset.

e.g. **tst.w R11**



Program Control Instructions

BR (emulated instruction) / JMP

- The BR instruction allows branch without any constraint to be performed
- The mnemonic is given by **br dst**
- Actual instruction is **mov dst, R0**
- In addition to the BR instruction, it is possible to jump to a destination using the unconditional jump instruction:

jmp dst

Conditional Jump Examples

Conditional jump - Jump if equal ($Z = 1$):

JEQ destination or JZ destination

label1:

MOV 0x0100, R5

MOV 0x0100, R4

CMP R4, R5

JEQ label1

- The above example compares the contents of registers R4 and R5;
- As they are equal, the flag Z is set, and therefore, the jump to address label1 is executed.

Conditional Jump Examples

- Conditional jump - Jump if different ($Z = 0$):

```
JNE destination or JNZ destination
```

```
label2:
```

```
    MOV    #0x0100, R5
```

```
    MOV    #0x0100, R4
```

```
    CMP    R4, R5
```

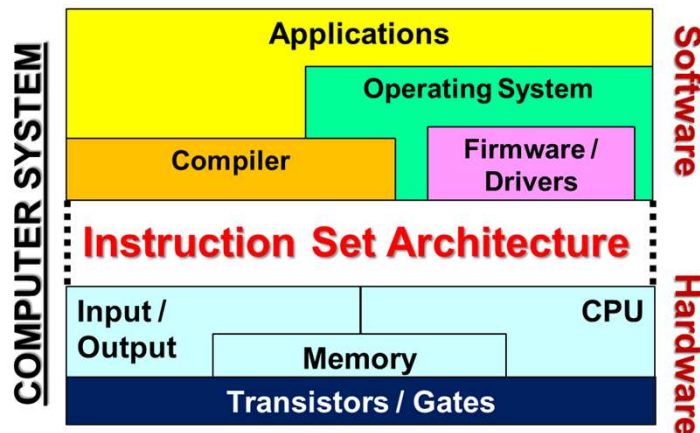
```
    JNZ    label2
```

- The above example compares the contents of registers R4 and R5;
- As they are equal, the flag Z is set, and therefore, the jump to address label2 is not executed.

Summary

LEARNING OUTCOMES OF CHAPTER:

✓ Understand and be able to use the core MSP430 executable instructions which can be grouped into the following categories: Data Movement, Arithmetic, Logical and Bit Manipulation, Rotation and Program Control instructions



**NEXT UP:
The Stack and
modular
programming**

